VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Policy-aware Semantic Genomic Data

Elias Crum[ORCID1,2*], Bart Buelens[ORCID2], Gökhan Ertaylan[ORCID2†], Ruben Taelman[ORCID1†]

[1]IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.
[2]Digital Biological Systems, Flemish Institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

*Corresponding author(s). E-mail(s): elias.crum@ugent.be;
†These authors share final authorship and jointly supervised this work.

Abstract

Background: The Variant Call Format (VCF) is compact, widely used, and readily parsed, but workflows that combine genomic observations with changing evidence, clinical context, or data-use policies rely on relationships maintained outside the file. Making such relationships explicit could support variant reanalysis, pharmacogenomic decision support, and consent-aware reuse, provided that the converted representation remains verifiable against the source. We present VCF-RDFizer, a reusable command-line tool that converts VCF into the Resource Description Framework (RDF) using the public VCF Core vocabulary. **Results:** We evaluated source fidelity, linked and policy-aware reuse, scalability, retrieval, and semantic coverage relative to four executable VCF-to-RDF converters. In the core benchmark's 78 validation runs, all 1,006 applicable comparisons between SPARQL results and answers computed directly from the source VCF agreed; on fault-injected graphs, the same source-comparison queries detected 96 of 113 injected faults, and deeper SHACL constraints detected the remaining 17. A linked workflow joined variant observations to ClinVar classifications in 81 ACMG secondary-finding genes and to population frequencies, and simulated consent policies decided which matches each data requester could receive. The linked workflow returned the same requester-specific matches as a conventional workflow using bcftools on gene-span slices of five heterogeneous VCFs, a 104-participant cohort, and two complete single-sample VCFs. On shared inputs,

VCF-RDFizer was the only one of five converters whose graphs matched the answers computed directly from the VCF for all eight content questions (record, genotype, and header summaries), although its broader representation increased conversion time and output size. Direct VCF processing was faster for one-pass analyses and for indexed regions up to 100 kb, whereas an indexed graph was faster for 10 Mb regions and, once about two minutes of conversion and indexing had been paid, after about 11 repeated whole-file questions.
Conclusions: VCF-RDFizer provides a verifiable and reusable semantic representation of VCF rather than a universal replacement for direct VCF processing. This study quantifies the additional representation and indexing costs of a semantic VCF representation. More generally, VCF-RDFizer is a first step toward applications that link genomic observations to external knowledge and data-use conditions.

Keywords: Genomic variation, Variant Call Format, RDF, SPARQL, Semantic validation, Data-use policies

1 Background

The Variant Call Format (VCF) is widely used to represent and exchange genomic variation in research and clinical sequencing, including pharmacogenomics and precision medicine [1–3]. VCF is compact, readily parsed, and already supports identifiers, reference information, and extensible annotations [1, 4]. However, many biomedical questions combine genomic observations with clinical records, external knowledge, sample metadata, and conditions governing data use [5–7]. The central challenge is therefore not reading VCF, but representing relationships that extend beyond the file in a form that can be reused across systems and questions. We consider three generalized use cases that illustrate this need.

(1) Clinical variant interpretation and reanalysis. Clinical interpretation combines genomic observations with clinical assertions, population frequencies, gene–disease associations, and patient phenotypes, all of which can change over time [8, 9]. Reanalysis can consequently identify new pathogenic or likely pathogenic findings and downgrade earlier classifications [10]. The genomic observation and the supporting evidence therefore follow different lifecycles, while the relationship between observation and evidence is often maintained only in annotations or application-specific databases.

(2) Pharmacogenomic clinical decision support. Pharmacogenomic decision support connects genomic observations to an interpreted phenotype, medication, indication, and clinical guidance. For example, CYP2C19-guided clopidogrel recommendations depend on both metabolizer phenotype and clinical indication [11]. Current systems still face challenges in representing genomic results as discrete data, integrating them with electronic health records, and maintaining the knowledge used for interpretation [12, 13].

(3) Consent-aware genomic data reuse. Secondary use adds a governance dimension to the same integration problem. Participant consent and associated data-use conditions define which research purposes are permitted, and those permissions

can vary across participants, datasets, and intended uses [7, 14]. The GA4GH Data Use Ontology (DUO) and Machine Readable Consent Guidance provide machine-readable vocabularies and structures for representing such conditions, while decisions about whether a specific use is authorized remain subject to institutional, ethical, and legal oversight [7, 14, 15]. Because genomic data are sensitive and potentially identifying, release decisions for participants, genomic regions, or individual variants remain tied to the consent and governance conditions under which the data may be used [16, 17].

None of these tasks is impossible with the current VCF representation. Variant-analysis frameworks, clinical systems, and data-access infrastructures already address important parts of the workflows [7, 12, 18]. The remaining challenges increasingly concern interoperability, reproducibility as knowledge changes, and the computational and operational cost of connecting independently maintained systems. VCF was designed to represent genomic variation, genotypes, and associated annotations rather than to serve as a shared model linking source observations to longitudinal evidence, clinical context, and data-use conditions [19]. Downstream systems therefore often add such relationships outside the source representation; for example, GEMINI imports VCF variants into a relational database and augments them with resources such as ClinVar, dbSNP, and population annotations to support integrated queries [18]. Such application-specific integration motivates a shared representation in which relationships between source observations and independently maintained information can be expressed explicitly and reused across workflows.

Knowledge graphs provide a way to connect entities and relationships across independently maintained data sources. RDF provides a standardized graph model in which resources are identified by IRIs, while Linked Data practices and SPARQL support cross-resource linking and querying [20–23]. The use of persistent identifiers and shared semantics also supports the interoperability and reuse goals of the FAIR principles, although RDF alone does not make data FAIR [24]. For genomic data, this allows a source observation to remain distinct from the biological variant, external evidence, clinical context, and data-use policy linked to it. SPDI (Sequence:Position:Deletion:Insertion) representations can support joins across files and resources such as dbSNP and ClinVar [25–27], while ODRL (the W3C Open Digital Rights Language) and DUO can represent policy conditions over identified resources [14, 28]. RDF therefore provides a reusable representation for integration, but reliable use still depends on defined identity rules, interpretation processes, and policy enforcement [28, 29].

Several semantic-genomics initiatives and VCF-to-RDF converters already explore this foundation. Many were developed for focused applications or as components of larger pipelines and consequently represent only the VCF information required for those tasks [30–34]. Existing tools also differ in their RDF models, treatment of headers and genotypes, and extension mechanisms [31, 33, 35–37]. Supplementary Table S1 summarizes documented capabilities and the evidence behind each assessment. Non-RDF frameworks such as Hail, GEMINI, and TileDB-VCF, and standards such as Beacon, htsget, and FHIR Genomics, address complementary parts of variant analysis, retrieval, and clinical exchange [18, 38–42]. Our aim is not to replace such systems

but to produce sufficiently complete RDF for multiple downstream links and queries without redesigning the representation for each application.

Producing RDF is only a first step. The representation should also remain verifiable against its source, and the practical cost of producing and using it should be characterized. SPHN Schema Forge, FAIR-Checker, and a FHIR RDF enrichment pipeline provide precedents for assessing transformations alongside the validation and information requirements of their intended workflows [43–45].

We present VCF-RDFizer, a tool that converts VCF into RDF using the public VCF Core vocabulary [46]. The vocabulary was developed separately [47], while the present study focuses on the converter, its representation choices, and the potential for downstream use of the generated RDF.

The evaluation of our tool investigates whether the RDF representation can preserve verifiable source information, support links to independently maintained knowledge, and enable policy-aware data reuse without modifying the converted VCF data. A linked research workflow combines genomic observations with ClinVar classifications, population frequencies, and simulated data-use policies [27]. Requester-specific query results are compared with an equivalent conventional workflow that operates directly on the VCF files, while separate experiments assess conversion fidelity, validation sensitivity, scalability, and retrieval costs.

Three research questions guide the evaluation:

RQ1: Which aspects of VCF information are preserved in RDF and can be validated against the source?

RQ2: How does the RDF representation support linking genomic observations to external knowledge and enable policy-aware querying?

RQ3: What are the computational costs of RDF representation choices, and when can the additional capabilities of VCF as RDF justify those costs?

Together, the experiments evaluate technical requirements shared by clinical variant reanalysis, pharmacogenomic decision support, and consent-aware genomic data reuse; they are intended to inspire future applications, not to offer directly implementable solutions.

2 Implementation

VCF-RDFizer is a command-line tool for converting VCF into RDF and validating selected properties of the result against the source file.[1] A pinned Docker image contains the conversion toolchain, and tagged versions are also distributed through PyPI and conda-forge [48–50].

2.1 Representation of VCF

VCF-RDFizer represents the source VCF document rather than only an interpreted biological variant. The VCF Core vocabulary separates file and header assertions, variant records, alleles, and sample/genotype assertions (Figure 1) [46, 47]. The vocabulary aligns VCF-specific structures with FALDO, the Sequence Ontology, HERO, and selected GA4GH VRS concepts [51–55].

[1] Source code and documentation: https://github.com/ecrum19/VCF-RDFizer.

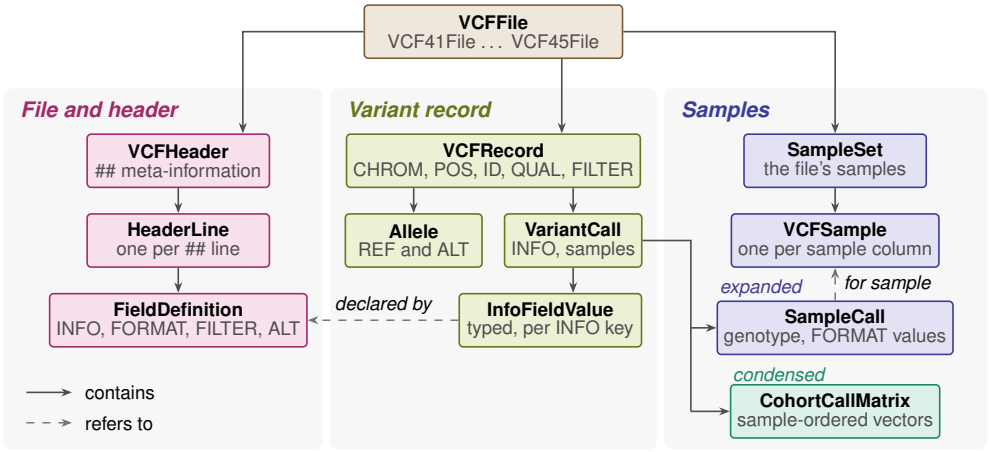


Fig. 1 Central VCF Core terms emitted by VCF-RDFizer. A versioned file contains its header, records, and sample set; a record contains alleles and a call; and the call holds INFO values and either expanded sample calls, one resource per sample, or condensed vectors, one ordered list of values per FORMAT field in sample order. Values retain links to the header declarations that define them. Supplementary Figure S1 presents the fuller model.

VCF-RDFizer assigns each VCF record a deterministic IRI derived from its source file identifier and record position. For example, two VCF files reporting the same genomic alteration receive distinct record IRIs because each identifier incorporates the originating file (see the VCF Core Vocabulary companion for an example). Each observation therefore retains its own quality scores, annotations, and provenance, while a separate linkset, a file of link triples kept apart from the converted graph, can connect both records to a shared variant identifier. Two sample profiles trade access granularity against graph size: *expanded* represents sample calls and FORMAT values as directly addressable resources, whereas *condensed* stores the same values in ordered vectors, for example one text value listing every sample's genotype for a record, which a query must split to reach a single sample. The reasoning for having separate sample profiles is due to performance considerations, and explained in-depth in Supplementary Section S3.1. Both profiles retain file, header, and record context. Supplementary Section S3 provides the complete model, capability matrix, and scaling derivation.

2.2 Conversion and validation

Conversion first derives tabular logical sources from the VCF (Figure 2). RML mappings, declarative rules that map tabular data to RDF, are executed by RMLStreamer and produce the record backbone of nine core triples per record, while direct emitters, code in the converter that writes triples without mappings, add typed values, header attributes, and sample structures that require per-value handling [56, 57]. The implementation is therefore hybrid: editing the RML mappings changes only the record backbone, not the sample, header, and typed-value triples written in code. Output is appended incrementally to plain or gzip-framed N-Triples. HDT (a compressed binary

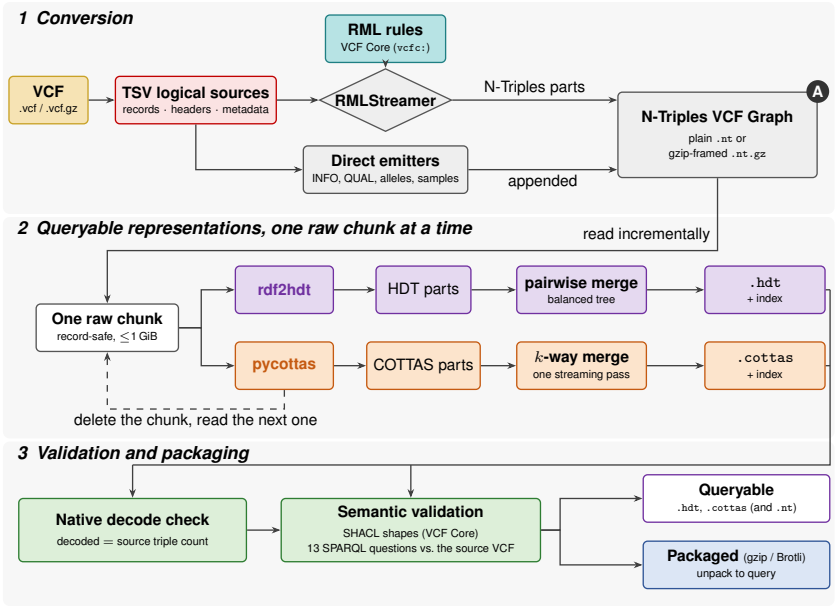


Fig. 2 The VCF-RDFizer pipeline. (1) Conversion: the VCF is split into tab-separated tables (logical sources); declarative RML mapping rules, executed by RMLStreamer, and the converter's own emitters turn them into RDF triples, appended incrementally as plain or gzip-framed N-Triples (one triple per line; gzip-framed output is a sequence of gzip members). (2) Optional queryable representations: HDT and COTTAS, compressed queryable formats, are built one chunk of complete N-Triples lines at a time before merging and indexing. (3) Validation and packaging: decoding checks on each artifact's triple count, the source-comparison queries Q1–Q13, and enabled SHACL profiles assess different properties of the result.

RDF format that can be queried without decompression), COTTAS (compressed triple tables stored as Parquet files), and additional gzip or Brotli compressions are optional artifacts rather than prerequisites for obtaining RDF [58–60]. Native decoding checks compare each HDT or COTTAS triple count with that of the N-Triples source graph. Construction and query-path details are provided in Supplementary Section S3.2.

Validation compares the results of 13 SPARQL *source-comparison queries* (Q1–Q13) with answers derived separately from the source VCF using cyvcf2 and, for exact FILTER text, bcftools [4, 19, 61]. The source-derived answers serve as the *oracle*. Q1–Q4 count records per contig and window, allele class, transition or transversion, and FILTER value; Q5–Q6 count genotypes and recomputed allele counts; Q7–Q8 compare file metadata and header lines; Q9–Q10 count triples per predicate and resources per class; and Q11–Q13 compare hashed summaries of record, INFO, and FORMAT values bound to source records (Supplementary Table S5). Normalized results are compared for exact agreement, although the value summaries are not exhaustive value-by-value tests. Preflight queries (14 integrity checks run before the comparisons) and SHACL constraints (declared shape rules that the graph must satisfy) assess additional graph properties [62]. The default profile checks cardinality and datatypes, while two deeper profiles check that related values agree (for example, GT text and called

alleles) and that indices are unique. Table 1 summarizes the assessment boundary, and Supplementary Section S4 specifies the queries, normalization, profiles, and limits.

Table 1 What VCF-RDFizer represents for each part of a VCF, which checks verify it, and what those checks do not establish. Q1–Q13 are the source-comparison queries (Section 2.2); Q9–Q10 inventory predicates and classes across the whole graph. Shapes are SHACL constraints, fixtures are small purpose-built test VCFs, and the linked workflow is evaluated in Section 3.2.

Information	Representation	Checked by	Limit
File and header	Versioned file, declarations, and linked metadata	Q7, Q8, shapes	Attribute–value agreement needs the deeper profiles
Record and alleles	Core fields, ordered alleles, typed and explicitly missing values	Q1–Q4, Q11, shapes	Summary digest, not exhaustive equality
INFO	Raw string plus optional typed, structured values	Q11 (raw), Q12, shapes	Preserved, not universally interpreted
Samples and FORMAT	Expanded sample calls or condensed vectors	Q5, Q6, Q13	Condensed values require decoding
VCF versions	VCF 4.1–4.5 file classes and retained newer fields	Fixtures	Some constructs retained lexically only
Links and policies	Separate linksets and policy-defined release views	Linked workflow	Identity assumptions and simulated policies

2.3 Linking and policy extensions

Two plug-ins extend the graph without modifying the converted source representation.

`vcf-rdfizer-link` executes linkers, declared rules that connect records or alleles to external identifiers, and writes the resulting relationships to separate linksets with coverage reports. The evaluated linkers connect eligible alleles to SPDI identifiers and variant calls to overlapping genes. The SPDI linker maps contig names such as `chr1` and `1` to the same reference sequence and minimally trims explicit alleles, removing bases shared by REF and ALT, so that the same simple variant in two files receives one SPDI identifier; it does not shift insertions or deletions within repeats, so equivalent alleles there can still receive different identifiers [25].

The third use case requires deciding, for each requester and purpose, which files, regions, or variants may be released. ODRL expresses such permissions and prohibitions as data, and DUO supplies standardized purposes. `vcf-rdfizer-policy` evaluates ODRL rules over files or declared selections, including regions, named variants, and gene-linked records [14, 28]. The evaluated profile applies default-deny semantics, so anything not explicitly permitted is withheld, and deny-overrides semantics, so a prohibition wins over a conflicting permission, with permitted purposes drawn from a subset of DUO. Ownership rules define which dependent resources are

removed with a withheld file or record; withholding a file, for example, also withholds its header, records, alleles, and sample calls. For each requester, the plug-in materializes a *release view*, a copy of the graph that contains only the triples that requester may receive, and checks the view against the policy and against a record list derived directly from the source VCF text. The evaluated plug-in supports requester-specific release under simulated policies, but does not anonymize genomic data or enforce additional policy obligations, such as requirements to delete released information [28, 63]. Operational deployment would additionally require authentication, access enforcement, auditing, and other safeguards for secure genomic data sharing [64]. Supplementary Section S5 describes linking tiers, policy evaluation, and release validation.

3 Evaluation and results

The experiments assess representation fidelity and validation (RQ1), a linked and policy-aware workflow (RQ2), and scalability, retrieval, and comparative semantic coverage (RQ3). Each subsection below first describes how its question was tested and then reports what the tests showed. The *core benchmark* comprises 143 runs in 12 experiment families covering conversion, scaling, storage, input coverage, configuration, validation, and query cost, all executed with the published VCF-RDFizer v3.1.0 release on two hosts with identical hardware. The linked workflow, the shared-input converter comparison, regional and large-graph retrieval, the index-storage measurement, an N-Triples-only rerun of the query-cost workload, and two later validation runs (a default-profile fault-injection rerun and the consumer WGS run) are separate experiments and are not counted in the core benchmark's totals. The N-Triples-only and fault-injection reruns used v3.1.0, and the other separate experiments used v3.3.1, which includes the evaluated linkers and the policy plug-in; on the same 100,000 HG005 records, both releases wrote identical sorted triples. Supplementary Section S6.1 lists the core benchmark's experiment families, and Section S6 provides the release ledger, hosts, dependencies, and archived evidence for every experiment. An interactive results browser, built from the same run records by the code that draws the figures, presents the measurements for each research question.[2]

The public VCF files used in the evaluation include Personal Genome Project donations, GIAB benchmark files, a precisionFDA HG002 file, the 2,504-sample 1000 Genomes chromosome-20 call set, and HGSVC2 structural variants [65–69]. HG005, a GIAB benchmark VCF, supports validation, scaling, retrieval, and the converter comparison; slices of the 1000 Genomes chromosome-20 call set support the sample-count series and the converter comparison; and the consumer whole-genome VCFs `NG131FQA1I` (Dante Labs) and `NB72462M` (Nebula Genomics) support the consumer WGS validation run and Scenario 4, respectively. The corpus experiment converts 250,000-record slices of eight public inputs and the complete HG005 VCF. Supplementary Table S14 lists the inputs.

[2] https://ecrum19.github.io/vcf-rdfizer-testing/

3.1 RQ1: Source comparisons and structural validation provide complementary evidence

3.1.1 Evaluation design

Each paired validation run converts a VCF, compares the Q1–Q13 answers on the resulting RDF with answers derived from that VCF, and applies SHACL where enabled. The inputs are 11 difficult-input fixtures, small VCFs with edge cases such as zero records, sites only, or a truncated gzip; a 1,000-line fixture converted under 10 option configurations and in the mode experiment; a 9,986-record fixture converted under both sample profiles and into N-Triples, HDT, and COTTAS; and HG005 slices of up to 100,000 records. A later consumer WGS validation run tests 250,000 records from `NG131FQA1I` with the default profile. A fault-injection experiment applies 113 deliberate corruptions and evaluates detection by queries alone, by the default profile, and by all three profiles. Separate retrieval runs compare the same 13 answers without SHACL on graphs derived from one million records and the complete HG005 VCF. Supplementary Section S4 defines the fault model, oracle dependencies, exclusions, and execution limits.

3.1.2 Results

All 78 validation runs in the core benchmark completed, on fixtures and slices up to 17.1 million triples. Across these runs, all 1,006 applicable SPARQL results agreed exactly with their source-derived counterparts after normalization. Eight additional genotype-query instances were not applicable because the zero-record and sites-only fixtures contain no genotypes. All 1,160 internal consistency checks between query results passed, for example that transitions plus transversions equal biallelic SNVs. To test whether answers depend on the query engine or storage format, 25 validation runs in the equivalence and query-cost experiments executed the same queries on four SPARQL engines: QLever, a SPARQL engine that builds its own index from the decoded graph, and three Comunica engines that query N-Triples, HDT artifacts, and, through a DuckDB-backed engine, COTTAS artifacts directly [70–74]. Every engine returned the same normalized answers. The other 53 runs used one engine: Comunica over the N-Triples in the difficult-input, covering-set, and mode experiments (44 runs), and QLever in the query-cost experiment (9 runs). Supplementary Section S6.1 describes each experiment's setup, and Sections S4.1 and S6.2 describe the engines and their pinned versions. All 152 HDT and COTTAS artifacts decoded to the same triple count as the corresponding N-Triples source graph.

Each validation layer establishes a different property. Query agreement covers the specified summaries, SHACL covers the enabled constraints, and decoding checks cover triple counts after serialization; none alone proves complete semantic equivalence. The separate complete-HG005 retrieval run and consumer WGS validation run are not included in the 1,006-comparison denominator. For the consumer WGS run, all 13 query results agreed and batched default-shape validation reported no violations on 58.2 million triples.

The source-comparison queries detected 96 of 113 injected faults. The other 17 changed relationships or values outside the query summaries, including called-allele

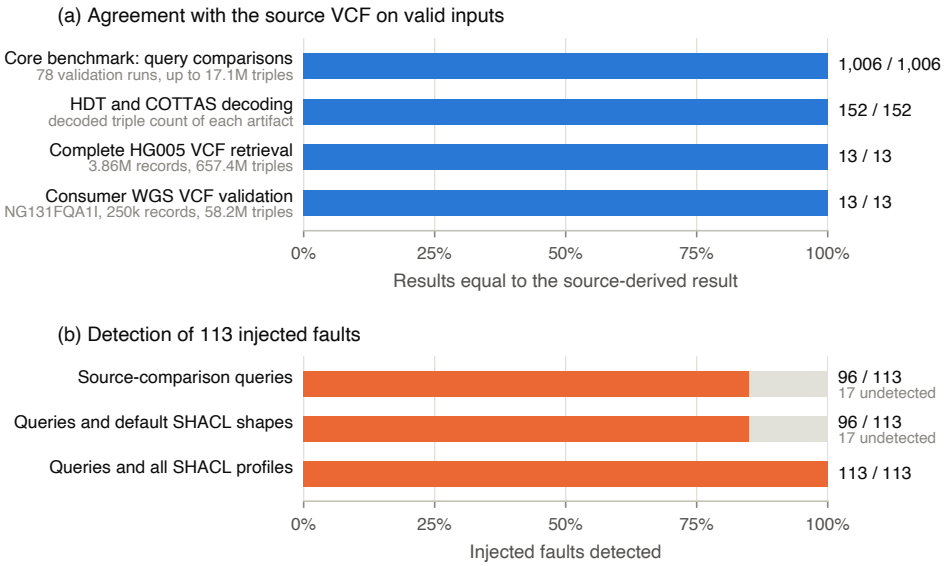


Fig. 3 Source agreement and detection of injected faults. Panel (a) summarizes source agreement in the core benchmark, native artifact decoding, and the separate complete-HG005 and consumer WGS query comparisons. Panel (b) shows detection of the 113 injected faults by source-comparison queries, the default SHACL profile, and all three profiles. The deeper SHACL profiles detected every injected fault that the source-comparison queries missed.

links, sample indexes, phasing, and header attributes. The default profile detected none of these 17 faults, while the two deeper profiles detected all 17 and accepted both uncorrupted control graphs. The deeper profiles took 33.7 and 73.2 seconds on this graph of about 2,000 triples, compared with 1.7 seconds for the default profile; because they also load the whole graph into memory, they were evaluated only on the small fault-injection graph.

Testing identified three implementation defects that record counts alone would not detect: Q9 and Q10 found missing sample resources, the SHACL shapes found dangling allele references, and the policy plug-in's release check found a withheld file's retained header; all were corrected. Fixtures covering VCF versions 4.1–4.5 converted successfully. Of 11 difficult inputs, 10 converted and validated, and a truncated gzip was correctly rejected; the sites-only result comes from a rerun after a defect in its test fixture was corrected. Supplementary Sections S7, S4, and S8 document the defects, test outcomes, and remaining limitations.

3.2 RQ2: Shared identifiers enabled reusable joins and policy-checked release views

3.2.1 Evaluation design

The three motivating use cases share one requirement: a genomic observation should remain traceable to its source while participating in relationships maintained elsewhere. The evaluation therefore tests one linked workflow rather than implementing the three applications in full. The application asks which participant–variant–gene matches connect an observed allele to a ClinVar classification in the fixed 81-gene ACMG SF v3.2 set and which matches each requester may receive [75]. A match is a participant–variant–gene tuple in which an observed allele equals a ClinVar record of any classification with at least one review star, so most matches are benign; pathogenic or likely pathogenic matches are reported separately. The workflow does not implement the additional criteria required for reportable clinical secondary findings.

The RDF workflow links converted VCFs to ClinVar and gene resources, materializes requester-specific views, and queries each view with SPARQL. The conventional workflow harmonizes contigs, annotates with bcftools, and applies the same case definition and simulated release rules in Python. The two workflows must return identical sorted participant, gene, contig, position, and allele tuples for every requester. Both workflows share preprocessing and the case definition, while downstream linking, policy evaluation, and query implementations remain separate.

Four evaluation scenarios test heterogeneous inputs, cohort breadth, complete-VCF scale, and fine-grained release. Scenario 1 contains gene-span slices, the records within the 81 gene spans, from five single-sample VCFs (HG002, HG004, HG005, `NG131FQA1I`, and `NB72462M`), Scenario 2 contains 104 unrelated high-coverage 1000 Genomes participants, Scenario 3 contains the complete HG005 VCF, and Scenario 4 contains the complete `NB72462M` VCF with four non-empty requester views [76]. Scenario 2 stores required population frequencies once in a separate sites-only graph rather than repeating panel-level INFO for each participant. Its rare-variant query joins participant observations, ClinVar classifications, and panel frequencies through SPDI, retaining matches with panel allele frequency below 0.01. All scenarios use the ClinVar release from 2026-09-13 [77]. Each requester has one purpose: clinical care (CC), disease-specific cardiovascular research (DS), general research use (GRU), and, in Scenario 4, care by the participant's physician (OP). Consent and access conditions are simulated and combine file permissions, withdrawals, gene panels, a region, and one named variant. For example, in Scenarios 1–3, records in the 28 cancer-predisposition genes are released only for clinical care, and HG005's simulated consent permits only clinical care, so its DS and GRU views in Scenario 3 are empty. Supplementary Section S5 specifies every scenario, policy, query, and comparison gate.

3.2.2 Results

Across all four scenarios, the RDF and conventional workflows returned identical requester-specific participant–variant–gene match sets (two examples in Figure 4; all four in Supplementary Figure S3). Every materialized release view passed the policy

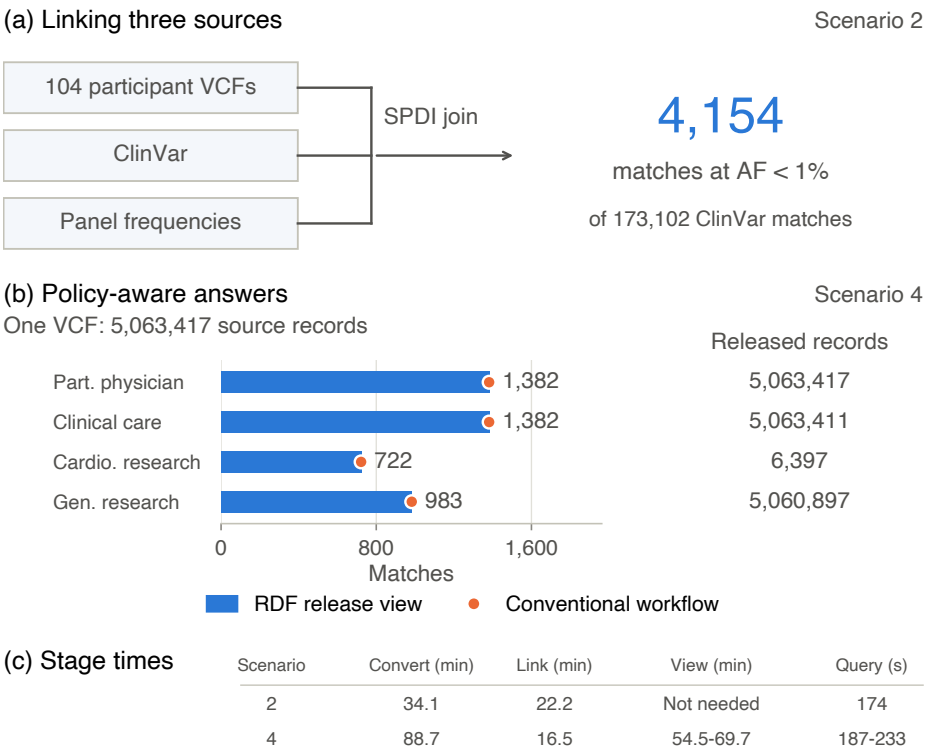
and source checks. Released-record counts also agreed in Scenarios 1, 3, and 4. In Scenario 2, however, the RDF workflow released records that the policy was meant to withhold: 110 symbolic structural-variant records in restricted cancer-predisposition genes to the disease-specific requester and 58 to the general-research requester. VCF-RDFizer v3.3.1 links no symbolic allele, such as `<DUP>`, to a gene, so gene-based rules could not select these records; no match changed, because the case definition joins only explicit alleles (Supplementary Section S5.3). The release checks could not detect these records, because they use the same gene links as the policy evaluator. For HG005, the query over the complete VCF reproduced the result from its gene-span slice exactly, returning 1,496 matches. Before policy filtering, the pathogenic or likely pathogenic subset contained no matches in Scenarios 1, 3, and 4 and eight in the Scenario 2 cohort (Supplementary Table S8). The counts describe matches under the study definition, not unique participants or clinically adjudicated findings.

Scenario 4 tested fine-grained release from the complete `NB72462M` VCF, containing 5,063,417 normalized records. The participant's physician received every record, clinical care received all but six APOE records, general research received all but 2,520 restricted records, and disease-specific research received only the 6,397 records in its cardiovascular panel. The two workflows agreed on both released-record counts and resulting matches.

The cohort rare-variant query demonstrates reuse of shared information. The workflow represents the required frequencies once in a 68,635-site graph of 6.5 million triples and joins observations, ClinVar, and frequencies through SPDI. Of 173,102 unrestricted ClinVar matches, 4,154 also had panel allele frequency below 0.01; both sets agreed exactly with the conventional workflow.

The two workflows were also compared on four applied rule changes (Supplementary Table S10). Shared variant identity changed how one of them was expressed. Restricting HFE C282Y to clinical care required only a six-line policy edit naming the variant's SPDI identity, with no change to the RDF policy engine, and because SPDI abstracts over contig naming, the edited rule also selected the HG002 record, whose contig name lacks the `chr` prefix. The conventional workflow required corresponding code and configuration changes. Three other rule changes were similarly small data edits in both workflows, so the result demonstrates reusable configuration for the tested identity-targeted rule rather than a general usability advantage.

Most linked-workflow cost occurred before query execution. For Scenario 3, converting the complete HG005 VCF and ClinVar required 55 minutes, linking required 8 minutes, and materializing, validating, and indexing the clinical view required a further 2.6 hours. Individual queries then required 216–218 seconds. Across all scenarios, queries over non-empty views took 171–233 seconds, which suggests that the fixed ClinVar join dominated the measured query time; the result does not establish scale-independent SPARQL performance. Scenario 4's selective views of a 1.2-billion-triple graph each required 55–70 minutes to materialize. After the preprocessing both workflows share, the conventional workflow annotated, selected, and released records for every requester in 4–91 seconds per scenario, whereas conversion and linking alone



(c) Stage times

Scenario	Convert (min)	Link (min)	View (min)	Query (s)
2	34.1	22.2	Not needed	174
4	88.7	16.5	54.5-69.7	187-233

Fig. 4 Linked RDF supports cross-resource queries and requester-specific genome access without changing the converted source data. Both scenarios use VCF-RDFizer v3.3.1. (a) In Scenario 2, shared SPDI identifiers connect 104 participant VCFs to ClinVar and a separately stored panel-frequency graph. Of 173,102 ClinVar-linked matches, 4,154 involve variants with a panel allele frequency (AF) below 0.01. The result demonstrates how independently maintained genomic resources can be combined in a single query. (b) In Scenario 4, simulated policies produce different release views from the same complete VCF containing 5,063,417 records. The participant's physician (part. physician) receives the complete file, whereas cardiovascular research (cardio. research) receives only 6,397 records. Blue bars represent RDF query results, and orange dots represent the conventional VCF workflow. Both workflows return identical participant–variant–gene tuples and agree on the number of released records for every requester. (c) Conversion and linking are performed once per scenario, while release views are generated separately for each requester. Scenario 2 reports the unrestricted rare-variant query and requires no release view. Scenario 4 reports the range of requester-specific view-writing and query times. Query times are medians of three executions. Indexing and release validation are excluded and reported in Supplementary Table S11. Together, the results demonstrate how RDF can support both data integration and selective reuse, although substantial preparation costs remain. Importantly, matching query results do not guarantee correct policy release: Scenario 2 retained 110 and 58 additional restricted records in the cardiovascular and general research views, respectively. Matches represent participant–variant–gene tuples, not clinical findings.

took 5.7–105 minutes in the RDF workflow, more than 40 times as long (Supplementary Table S11). Supplementary Figure S4 and Table S11 report every stage and requester.

3.3 RQ3: Semantic coverage and practical cost depend on representation and workload

3.3.1 Evaluation design

A shared-input comparison evaluates four other executable converters: JVarkit, TogoVar, SPARQLing Genomics, and BioInterchange. Each converter processes a 100,000-record HG005 slice and a 10,000-record, 16-sample 1000 Genomes slice with its most complete available options. Content questions Q1–Q8 are translated to each vocabulary and compared with the same source-derived oracle, normalization, comparator, and QLever build used for VCF-RDFizer. A question is reported as not represented when the graph lacks the required information. Conversion time, peak memory, and output size are measured over three replicates in pinned containers. Supplementary Section S2 provides the converter versions, execution commands, adapted SPARQL queries, and complete results.

Controlled scaling series vary one dimension at a time. The record-count series processes 10,000, 100,000, one million, and all HG005 records, while the sample-count series holds 10,000 cohort records fixed and varies the sample count from one to 2,504 under both profiles. Stage timings, resident memory, transient workspace, and retained artifact sizes are measured separately. Replicated scaling-series measurements are distinguished from single complete-VCF and corpus runs.

Retrieval experiments compare indexed QLever queries with full VCF parsing using cyvcf2. For five regional questions, the comparison also includes tabix-indexed VCF access through cyvcf2 and bcftools. Individual-question timings measure each question separately, requiring a full VCF parse for each question on the non-indexed VCF path. Batch timings measure all 13 questions together, requiring only one full VCF parse. Regional questions select records by genomic position (POS) within windows ranging from 1 kilobase (kb) to 10 megabases (Mb), using identical selection criteria across execution paths. Setup costs are reported separately from query execution. RDF setup includes conversion and index construction, whereas VCF setup includes bgzip compression and tabix indexing.

3.3.2 Results

On the shared inputs, VCF-RDFizer was the only converter whose graphs matched the source-derived oracle for all eight cross-vocabulary content questions (Figure 5). The other outputs differed because information was absent or transformed, not because different query engines were used. JVarkit merged five pairs of cohort records that shared chromosome, position, and reference allele. SPARQLing Genomics represented only calls carrying an alternate allele and therefore omitted homozygous-reference cohort calls. TogoVar represented allele-frequency data but not samples or headers and omitted symbolic and spanning-deletion alleles. BioInterchange answered all questions

on the cohort input, but invalid JSON in the HG005 header output prevented recovery of Q7 and Q8.

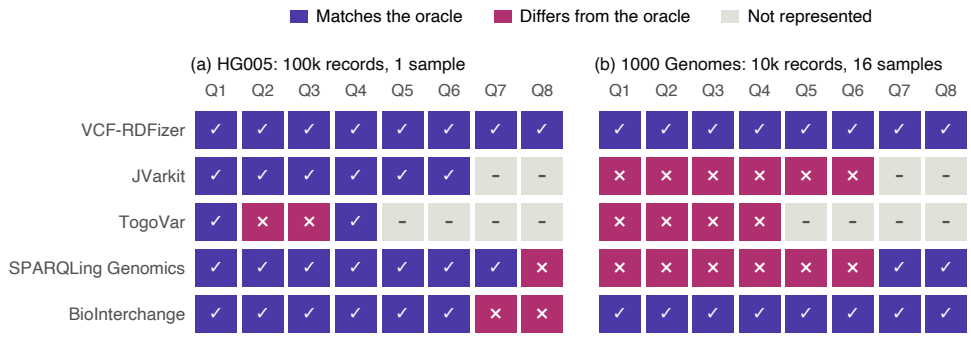


Fig. 5 Cross-vocabulary content-question results on two shared inputs. Each cell compares one ported question with the answer computed directly from the VCF: a check mark denotes agreement, a cross disagreement, and a dash information not represented in the graph. Q1–Q4 summarize records, Q5–Q6 genotypes, and Q7–Q8 the header.

Broader coverage carried additional cost. The other converters produced 14–57 triples per record on HG005 and 18–203 on the 16-sample input, and completed each input in 0.1–6.3 seconds, whereas VCF-RDFizer produced 171 triples per HG005 record and 536 per record on the 16-sample input, and took 31–92 seconds per input, including container startup for each stage. Supplementary Section S2 reports output sizes, memory, and per-question causes.

The record-count series produced 170.5–171.9 triples per VCF record, with mapping-stage resident memory between 1.0 and 1.7 GB as record count increased. Wall time and disk use grew with the data, while the observed mapping memory remained bounded over the tested range. Sample representation, not record count, determined the number of triples per record. At 10,000 records and 2,504 samples, expanded output contained 627.4 million triples and a 4.10 GB HDT, compared with a 59.2 MB HDT for condensed output. The condensed profile reduces repeated graph structure but requires vector-aware querying and validation. Supplementary Figure S6 gives the complete sample-count series.

Optional artifacts dominated one whole-VCF benchmark. The conversion of the complete HG005 VCF required 16.07 hours when both HDT and COTTAS were built, of which approximately 14.5 hours were spent constructing these compressed representations. Retaining HDT, its index, and COTTAS required 10.88 GB, while gzip-framed N-Triples alone required 2.89 GB.

On the 17.1-million-triple graph of 100,000 HG005 records, individual indexed QLever queries required 0.005–4.62 seconds, compared with 11.5–12.3 seconds for full cyvcf2 parses. For the 13-question batch, one cyvcf2 pass required 12.4 seconds, while the QLever queries required 17.1 seconds, excluding setup. The minimal RDF setup, which converts the records to gzip-framed N-Triples (75.8 MB) and builds a QLever index (214.5 MB) without HDT or COTTAS, required about two minutes; under the

tested repeated-question workload, the cumulative times crossed after approximately 11 separately executed questions, because each question saved about 11 seconds (in the N-Triples-only rerun, a median of 12.28 seconds per full parse against a mean of 1.43 seconds per indexed query). The N-Triples were 21–25 times larger than the gzipped VCF.

Regional queries give the fairer comparison, because the VCF side can also use an index and read only the requested region (Figure 6). Across 11,160 executions, all answers agreed and no execution failed. Tabix-indexed VCF access was faster for windows up to 100 kb and for 1 Mb windows with bcftools, while QLever was faster for 10 Mb windows and for 1 Mb windows with cyvcf2. QLever query time remained nearly constant across window sizes, whereas VCF-region retrieval time increased with the number of records parsed. Building the QLever index from existing N-Triples required 23.3 seconds, and bgzip and tabix required 0.46 seconds. The QLever index required 60.5 times the storage of the bgzipped VCF with its tabix index (214.5 MB against 3.5 MB), and 57.5 times for the complete HG005 VCF (8.1 GB against 140.9 MB).

On the 657.4-million-triple graph of the complete HG005 VCF, QLever answered all 13 questions in 646.4 seconds after an 880.8-second index build, with every answer equal to the parser-derived result. Loading the same graph from N-Triples, HDT, or COTTAS changed indexed query time by only 0.4%, although decode costs differed. When Comunica queried the HDT and COTTAS files directly, without a QLever index, it did not complete the full workload on the 171-million-triple graph of one million HG005 records. Supplementary Figure S8, Table S17, and Section S9 report the detailed engine and artifact results.

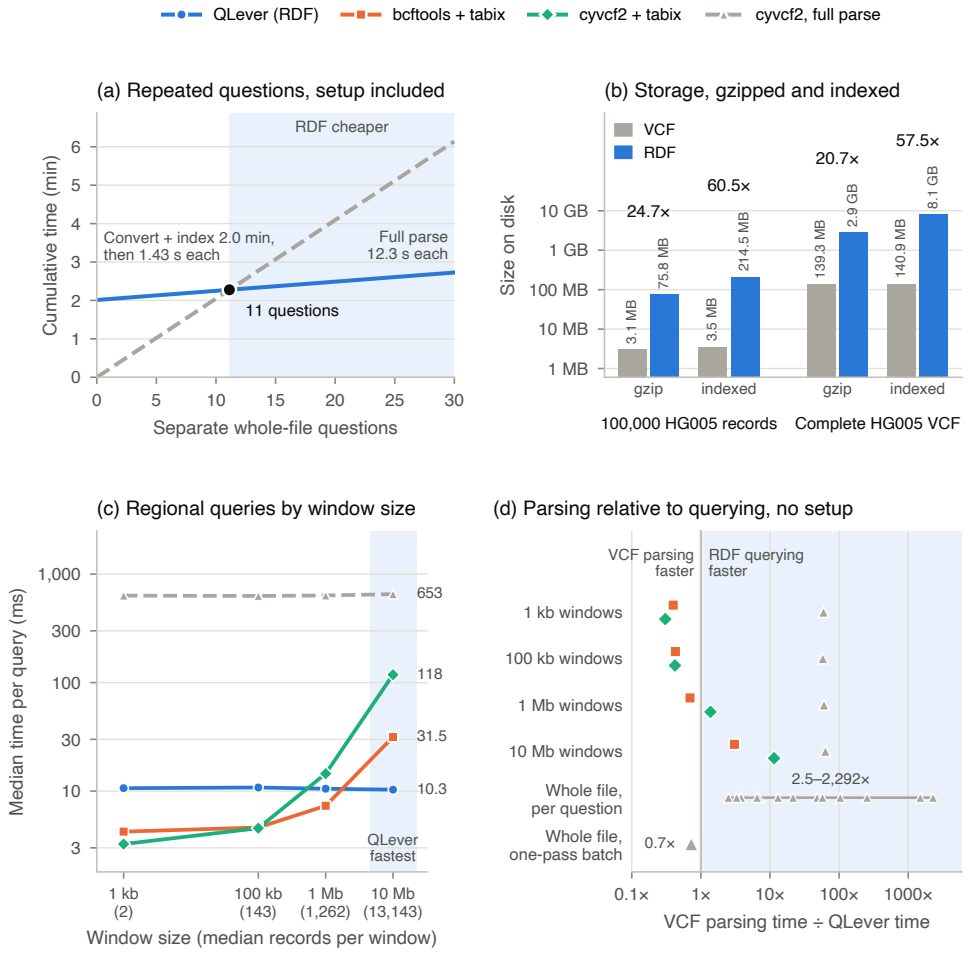


Fig. 6 Indexed RDF querying can offset conversion costs through repeated queries, while VCF parsing remains faster for narrow regions and one-pass analyses. Data: 100,000 HG005 records (17.1 million triples) unless noted; a whole-file question is one of the 13 source-comparison queries, and the five regional questions are window-restricted variants of Q1–Q5. (a) Including approximately two minutes for RDF conversion and indexing, cumulative RDF time becomes lower than that of repeated VCF scans after 11 separately executed questions. (b) Gzip-framed N-Triples require 20.7–24.7 times more storage than the corresponding gzip-compressed VCF; the QLever index requires 57.5–60.5 times the storage of the bgzipped VCF with its tabix index. (c) Regional queries show nearly constant QLever execution times as window size increases from 1 kb to 10 Mb, while indexed VCF retrieval becomes slower for larger windows. (d) Ratios of VCF parsing time to QLever query time, excluding setup, show which approach performs better across workloads; values above one favor QLever. Together, the results show that repeated and large-region queries can offset RDF's additional processing time, whereas direct VCF access remains advantageous for regions up to 100 kb and when all questions are answered in one VCF pass.

4 Discussion

This section first relates the evaluated representation to the three motivating use cases (RQ2; Sections 4.1–4.4). It then discusses what the validation results establish (RQ1; Section 4.5) and when the measured costs favor RDF or direct VCF processing (RQ3; Section 4.6), and closes with the scope of the study and next steps.

4.1 A first step toward reusable genomic information

Clinical reanalysis, pharmacogenomic decision support, and consent-aware genomic reuse require more than converting VCF into another serialization. Each application depends on explicit links between source observations and information maintained elsewhere, including evidence, clinical context, and conditions governing data use. VCF-RDFizer provides an evaluated representation layer by retaining source context, validating selected properties of the RDF, and demonstrating integration and policy-aware release. The study evaluates that foundation rather than the clinical effectiveness, operational utility, or security of complete applications.

Agreement with the conventional workflow confirms that the demonstrated task is not exclusive to RDF. The distinction is representational: observations, external resources, shared identifiers, and policies are expressed in a common graph that can be queried and extended without rewriting the source representation. The separate frequency graph and identity-targeted policy demonstrate reuse within the tested workflow, but do not establish a general reduction in development effort or imply that conventional pipelines cannot provide equivalent functionality.

4.2 Clinical interpretation and reanalysis

In the evaluated workflow, each converted VCF was joined to a fixed ClinVar release through SPDI identifiers, and the workflow returned the same matches as the conventional workflow in every scenario. RDF could make the evidence supporting an interpretation explicit and inspectable. A genomic observation could remain linked to the assertions used at a specific time, while later evidence is added with separate provenance rather than replacing the earlier interpretation. Queries could then identify observations affected by a revised clinical assertion or frequency estimate and nominate the corresponding cases for review [8, 9]. Such a workflow would support reevaluation without replacing clinical interpretation or recovering variants absent from the original call set.

The evaluated ClinVar and frequency joins demonstrate static relationships required by such a workflow, not longitudinal reanalysis. A practical service would require validated rules for assemblies, allele normalization, and unresolved matches. Minimally trimmed SPDI expressions do not necessarily correspond to NCBI contextual or canonical alleles in repetitive regions, so equivalent alleles can receive different identifiers and miss evidence joins [25]. For example, a one-base deletion in a run of identical bases can be written at any position in the run; NCBI's contextual SPDI spans the whole run, whereas minimal trimming keeps the position written in the VCF. Longitudinal evidence would also require assertion-level versioning, disease context, review status, provenance, relevant dates, and links to patient phenotypes, family

information, and prior reports. A change-detection process could then route affected observations for clinical review. Evaluation should compare nominated cases with conventional reannotation and expert-reviewed outcomes, including missed updates and unnecessary reviews. The clinical value reported in reanalysis studies cannot be inferred from the static query agreement measured here [10].

4.3 Pharmacogenomic clinical decision support

The evaluated linkers connected variant calls to overlapping genes and alleles to shared identifiers, the kind of link a pharmacogenomic decision-support path requires, although no pharmacogenomic interpretation was evaluated. RDF could represent the path from a genomic observation to a pharmacogenomic interpretation, medication order, indication, and prescribing guidance. Separate linked resources could preserve which observation, interpretation, and guideline version support a recommendation and allow an existing interpretation to be reused when the prescribing context changes. The model addresses an integration challenge reported in pharmacogenomic decision-support research without replacing established clinical systems or interpretation tools [12, 13].

The present workflow does not infer diplotypes, metabolizer phenotypes, or treatment recommendations. A practical implementation would connect represented calls to a validated interpretation pipeline, such as an appropriately configured PharmCAT workflow, and retain assumptions, evidence, and unresolved results [2]. The input would need adequate coverage, phasing, and clinically relevant variants; absence of a VCF record cannot automatically be interpreted as a reference genotype. Clinical deployment would then link the interpreted result to the correct patient, medication, indication, and context through agreed identifiers and exchange models, including FHIR Genomics where appropriate [42]. Computable guidance would require versioned rules and explicit handling of missing or conflicting information. The CYP2C19–clopidogrel example illustrates why phenotype alone does not determine a recommendation [11]. Expert validation, comparison with source guidance, and evaluation of the clinical workflow would remain necessary [12].

4.4 Consent-aware genomic reuse

RDF could make the relationship between a genomic resource, governing policy, and request purpose directly inspectable. Policies could target files, participant records, or declared subsets without creating a separate source copy for every permitted use. The evaluated plug-in demonstrates part of this model: requester-specific match sets matched the conventional workflow, released records agreed in three of the four scenarios, ownership checks detected dependent context, and Scenario 4 produced four distinct non-empty views from one complete VCF. The evaluation does not cover selective masking within a multi-sample VCF, and coordinate-based region rules still depend on source contig labels.

Operational deployment would require reviewed mappings from consent and institutional conditions to a defined policy profile. DUO and the GA4GH Machine Readable Consent Guidance support consistent terminology, but a computable term

does not determine whether a request is ethically or legally authorized [7, 14, 15]. A production profile would need agreed semantics for purposes, permissions, prohibitions, conflicts, and resource ownership. Default-deny and deny-overrides are choices of the evaluated implementation rather than guarantees provided by RDF or ODRL [28].

Policy evaluation would also need authenticated requester attributes and an enforcement boundary protecting source graphs, indexes, caches, exports, and query interfaces [7, 29]. Policies, decisions, and released views would require auditable versions. Withdrawals could control future access but would not retract previously exported copies. Security, privacy, and regulatory assessments therefore remain distinct from source-fidelity and policy-evaluation tests. The current plug-in demonstrates inspectable release decisions under simulated rules, not anonymization, regulatory assurance, or protection outside the evaluator.

4.5 Validation requires both source and graph checks

The most informative validation result is not the number of passing checks but that the deeper structural constraints detected every injected fault that the source comparisons missed. Source-comparison queries detected changed values and summaries, while the deeper profiles detected relationships that the current oracle does not inspect. Two of the three implementation defects found during testing, one caught by Q9 and Q10 and one only by the shapes, show why both layers matter. At the same time, a passing summary, shape profile, or triple-count check establishes only the property tested. The complete fault-detection result applies to the constructed 113-fault set on a small graph, and the deeper profiles have not been shown to be practical at whole-VCF scale.

Oracle quality is also part of validation quality. Expected predicate and class inventories depend on the selected representation, and the converter and oracle can share an incorrect assumption. Future validation should extend affordable checks for allele links, phasing, sample indexes, and header relationships, and compare values in ways that are insensitive to permitted lexical differences. No experiment establishes biological correctness of the original calls or exhaustive semantic equivalence between VCF and RDF.

4.6 Representation and workload determine the deployment choice

Semantic representation introduces real storage and setup costs. The expanded profile preserves directly addressable sample calls, whereas the condensed profile reduces repeated graph structure by encoding sample values in vectors. The condensed profile consequently lowers graph size at the cost of direct sample-level querying, validation, and policy targeting. Triple count alone does not capture the capability of either representation.

Direct VCF access remains preferable for many one-pass analyses and small indexed regions because no semantic conversion or graph indexing is required. The additional RDF setup may become beneficial when the same genomic data support repeated questions, independently maintained links, or policy-dependent release. A

clinical reanalysis service, for example, could retain a converted VCF while querying successive evidence releases instead of reconstructing every relationship for each cycle. The present experiments demonstrate prerequisites for such reuse but do not benchmark longitudinal updates.

The crossover after about 11 questions is specific to this workload, not a property of the tool: it assumes a full VCF parse for each question, the tested question mix, and the measured minimal RDF setup. If all 13 questions are known in advance, a single cyvcf2 pass answers them in 12.4 seconds, less than the 17.1 seconds of QLever queries even before setup, so RDF does not pay off for that workload. HDT and COTTAS should similarly be generated only when their separate storage or access properties are required. Applications with changing evidence or policies would also incur update costs for links, indexes, and release views that were not measured here.

4.7 Scope and next steps

The shared-input converter comparison covers four converters, two inputs, and Q1–Q8. The result establishes comparative content coverage under the tested configurations, not universal superiority or suitability for every purpose. The sample-count series held INFO content fixed, the complete multi-sample cohort was not converted, and the deeper SHACL profiles were evaluated only on small inputs. The linking experiments use single-sample files and simulated policies, and all outcomes remain tied to the recorded data, software, reference releases, and policy configurations. Gene-based policy selection also remains incomplete for some symbolic and breakend records, as documented in Supplementary Section S8.

Future work can proceed in stages. The first priority is to close remaining oracle gaps and define stronger boundaries for normalization and shared variant identity. Application-specific pilots can then evaluate versioned evidence updates, validated pharmacogenomic interpretations linked to clinical context, and selective release under reviewed policy models. Each pilot should involve the relevant domain experts and an application-specific comparator. Clinical deployment would additionally require governance, privacy, security, and human-workflow assessments appropriate to the intended environment. GA4GH VRS identifiers, federated querying, and policy evaluation over multi-sample data remain promising extensions.

VCF-RDFizer provides a testable starting point for this progression: a genomic representation that can be validated against its source, linked to external information, and reused across workflows while the additional requirements of each application remain explicit.

5 Conclusions

VCF-RDFizer makes VCF information explicit in a representation that can be validated against its source, linked to external knowledge, and used in policy-aware research queries. The evaluation demonstrates complementary validation, reusable joins, and requester-specific agreement with a conventional workflow, while showing that semantic coverage adds conversion, storage, and indexing costs. The resulting representation is therefore most relevant when persistent links, repeated questions,

or inspectable release rules justify those costs, rather than as a universal replacement for direct VCF processing. VCF-RDFizer is a first step toward evidence-aware reanalysis, pharmacogenomic decision support, and consent-aware reuse; complete applications still require validated interpretation, clinical integration, and enforced access processes.

Availability and requirements

Project: VCF-RDFizer. **Home page:** https://github.com/ecrum19/VCF-RDFizer. **Language:** Python, with the mapping, compression, and query tools specified in Supplementary Section S6. **Platforms:** Linux, macOS, and Windows are covered by the tool's tests; the experiments reported here ran on Ubuntu. **Distribution:** PyPI, conda-forge, and Docker. **License:** MIT. **Any restrictions to use by non-academics:** none. Docker supplies the pinned benchmark environment; requirements for other installations are documented with each release. Version 3.1.0 produced the core benchmark results and the N-Triples-only and fault-injection reruns, and v3.3.1 produced those of the linked workflow, regional and large-graph retrieval, the index-storage measurement, the consumer WGS validation run, and the converter comparison. On the same 100,000 HG005 records, both releases wrote identical sorted triples. Supplementary Section S6.2 records the build behind each result.

Declarations

Ethics approval and consent to participate

This study used publicly released genomic data and involved no recruitment of or interaction with participants. Personal Genome Project donors released their files under the project's open-consent model [65]; GIAB and precisionFDA files are public reference materials [66, 69]; the 1000 Genomes and HGSVC2 datasets are released for research reuse [67, 68, 76]; and ClinVar is a public NCBI resource [77]. The experimental consents are simulated and do not express any real participant's wishes. No identification of participants was attempted. Ethics approval was not sought for the reported work.

Consent for publication

Not applicable. The manuscript reports aggregate results and software measurements, not individual-level genomic records, images, or personal details.

Availability of data and materials

Input filenames, provider URLs, download commands, fixtures, derivation scripts, query files, manifests, normalized outputs, and comparison reports are maintained at https://github.com/ecrum19/vcf-rdfizer-testing. The benchmark harness and the run records behind every reported result are archived on Zenodo at https://doi.org/10.5281/zenodo.23301185; version v1.0.0, which holds the results reported here, is https://doi.org/10.5281/zenodo.23301186 [78]. Large source VCFs

are obtained from their public providers rather than redistributed in Git. Experiment 17 (`benchmarks/use_case/acmg`) contains the use-case definitions, simulated policies, conventional baseline, and applied-change tests. Its match sets, timings, and check reports, as well as the additional validation runs, are recorded under `benchmark-results`. The main branch holds the runs behind the reported results; pre-release, superseded, and failed runs, including those cited as defect evidence in the Supplementary Material, are archived on the repository's `legacy` branch. No biological materials were generated or used.

The software is available at the project home page. The core benchmark and the N-Triples-only and fault-injection reruns used v3.1.0 (`d3b34d5`; image `sha256:1904e96d...34aa`). The linked workflow, regional and large-graph retrieval, index-storage measurement, converter comparison, and consumer WGS validation run used v3.3.1 (`b25fb7b`; image `sha256:3ad71b1a...2993`), a release that includes the evaluated linkers and the policy plug-in. Supplementary Section S6 identifies each experiment's build, the image digests, and the bundled shape snapshot. The public vocabulary is separately versioned [46]. Results must be traced to their recorded build and input, not only to the latest package.

Competing interests

None declared.

Funding

Elias Crum acknowledges support from the Research Foundation–Flanders (FWO), SB Fellowship 1S27825N. This work was supported by the European Union under the Horizon Europe programme through the REALM project (grant agreement No. 101095435), the European Union's Horizon Europe research and innovation programme through the RAIDO (Reliable AI and Data Optimisation) project under grant agreement No. 101135800, and the Belgian Federal Public Service Policy & Support (BOSA) through the DigiTwin4PH (Digital Twins for Personalised Healthcare) project. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the granting authorities.

[AUTHOR QUERY: Complete the funding statements for B.B., R.T., and G.E.]

Authors' contributions

E.C., G.E., and R.T. conceived the study. E.C. did the implementation and coding work. E.C. devised the validation and use-case demonstration with the guidance of R.T. and executed the experiments. E.C., G.E., and R.T. reviewed and verified all results. E.C., B.B., G.E., and R.T. wrote and edited the manuscript. G.E. and R.T. share final authorship and jointly supervised this work.

Acknowledgments

Benchmarking used the resources of IDLab at Ghent University. Claude Opus 5.5 [79] and GPT-6 Astra [80] assisted with experiment automation scripting, result data analysis scripting, and restructuring and language editing of this manuscript. The authors reviewed all AI outputs and take full responsibility for the manuscript, experiments, tools, and repositories.

References

[1] SAM/BAM Format Specification Working Group: The Variant Call Format Specification (2024). https://github.com/samtools/hts-specs

[2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). Clinical Pharmacology & Therapeutics **113**(5), 1036–1047 (2023) https://doi.org/10.1002/cpt.2790

[3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeshirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nabbout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. European Journal of Human Genetics **30**(9), 1017–1021 (2022) https://doi.org/10.1038/s41431-022-01113-x

[4] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. GigaScience **10**(2), 008 (2021) https://doi.org/10.1093/gigascience/giab008

[5] Gottesman, O., Kuivaniemi, H., Tromp, G., Faucett, W.A., Li, R., Manolio, T.A., *et al.*: The Electronic Medical Records and Genomics (eMERGE) Network: past, present, and future. Genetics in Medicine **15**(10), 761–771 (2013) https://doi.org/10.1038/gim.2013.72

[6] The All of Us Research Program Genomics Investigators: Genomic data in the All of Us research program. Nature **627**(8003), 340–346 (2024) https://doi.org/10.1038/s41586-023-06957-x

[7] Rehm, H.L., *et al.*: GA4GH: International policies and standards for data sharing across genomic research and healthcare. Cell Genomics **1**(2), 100029 (2021) https://doi.org/10.1016/j.xgen.2021.100029

[8] Richards, S., Aziz, N., Bale, S., Bick, D., Das, S., *et al.*: Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. Genetics in Medicine **17**(5), 405–424 (2015) https://doi.org/10.1038/gim.2015.30

[9] Deignan, J.L., Chung, W.K., Kearney, H.M., Monaghan, K.G., Rehder, C.W., Chao, E.C.: Points to consider in the reevaluation and reanalysis of genomic test results: a statement of the American College of Medical Genetics and Genomics

(ACMG). Genetics in Medicine **21**(6), 1267–1270 (2019) https://doi.org/10.1038/s41436-019-0478-1

[10] Hiatt, S.M., Amaral, M.D., Bowling, K.M., Finnila, C.R., Thompson, M.L., Gray, D.E., Lawlor, J.M.J., Cochran, J.N., Bebin, E.M., Brothers, K.B., East, K.M., Kelley, W.V., Lamb, N.E., Levy, S.E., Lose, E.J., Neu, M.B., Rich, C.A., Simmons, S., Myers, R.M., Barsh, G.S., Cooper, G.M.: Systematic reanalysis of genomic data improves quality of variant interpretation. Clinical Genetics **94**(1), 174–178 (2018) https://doi.org/10.1111/cge.13259

[11] Lee, C.R., Luzum, J.A., Sangkuhl, K., Gammal, R.S., Sabatine, M.S., Stein, C.M., Kisor, D.F., Limdi, N.A., Lee, Y.M., Scott, S.A., Hulot, J.-S., Roden, D.M., Gaedigk, A., Caudle, K.E., Klein, T.E., Johnson, J.A., Shuldiner, A.R.: Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update. Clinical Pharmacology & Therapeutics **112**(5), 959–967 (2022) https://doi.org/10.1002/cpt.2526

[12] Wake, D.T., Smith, D.M., Kazi, S., Dunnenberger, H.M.: Pharmacogenomic clinical decision support: A review, how-to guide, and future vision. Clinical Pharmacology & Therapeutics **112**(1), 44–57 (2022) https://doi.org/10.1002/cpt.2387

[13] Johnson, D., Del Fiol, G., Kawamoto, K., Romagnoli, K.M., Sanders, N., Isaacson, G., Jenkins, E., Williams, M.S.: Genetically guided precision medicine clinical decision support tools: a systematic review. Journal of the American Medical Informatics Association **31**(5), 1183–1194 (2024) https://doi.org/10.1093/jamia/ocae033

[14] Lawson, J., Cabili, M.N., Kerry, G., *et al.*: The Data Use Ontology to streamline responsible access to human biomedical datasets. Cell Genomics **1**(2), 100028 (2021) https://doi.org/10.1016/j.xgen.2021.100028

[15] Global Alliance for Genomics and Health: Machine Readable Consent Guidance (MRCG). Technical implementation guidance, version 1.0. Accessed 7 October 2026. https://www.ga4gh.org/product/machine-readable-consent-guidance/ Accessed 2026-10-07

[16] European Parliament and Council of the European Union: Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88 (2016). https://eur-lex.europa.eu/eli/reg/2016/679/oj

[17] Shabani, M., Borry, P.: Rules for processing genetic data for research purposes in view of the new EU General Data Protection Regulation. European Journal of Human Genetics **26**(2), 149–156 (2018) https://doi.org/10.1038/s41431-017

-0045-7

[18] Paila, U., Chapman, B.A., Kirchner, R., Quinlan, A.R.: GEMINI: integrative exploration of genetic variation and genome annotations. PLoS Computational Biology **9**(7), 1003153 (2013) https://doi.org/10.1371/journal.pcbi.1003153

[19] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. Bioinformatics **27**(15), 2156–2158 (2011) https://doi.org/10.1093/bioinformatics/btr330

[20] Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1: Concepts and Abstract Syntax (2014). https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/

[21] Hogan, A., Blomqvist, E., Cochez, M., D'Amato, C., Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. ACM Computing Surveys **54**(4) (2021) https://doi.org/10.1145/3447772

[22] World Wide Web Consortium (W3C): LinkedData. https://www.w3.org/wiki/LinkedData Accessed 2026-03-04

[23] Harris, S., Seaborne, A., Prud'hommeaux, E.: SPARQL 1.1 Query Language (2013). https://www.w3.org/TR/2013/REC-sparql11-query-20130321/

[24] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J.G., Groth, P., Goble, C., Grethe, J.S., Heringa, J., Hoen, P.A.C., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., Schaik, R., Sansone, S.-A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., Lei, J., Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B.: The fair guiding principles for scientific data management and stewardship. Scientific Data **3**, 160018 (2016) https://doi.org/10.1038/sdata.2016.18

[25] Holmes, J.B., Moyer, E., Phan, L., Maglott, D., Kattman, B.: SPDI: data model for variants and applications at NCBI. Bioinformatics **36**(6), 1902–1907 (2020) https://doi.org/10.1093/bioinformatics/btz856

[26] Sherry, S.T., Ward, M., Kholodov, M., Baker, J., Phan, L., Smigielski, E.M., Sirotkin, K.: dbsnp: the ncbi database of genetic variation. Nucleic Acids Research **29**(1), 308–311 (2001) https://doi.org/10.1093/nar/29.1.308

[27] Landrum, M.J., Lee, J.M., Benson, M., Brown, G.R., Chao, C., Chitipiralla, S., Gu, B., Hart, J., Hoffman, D., Jang, W., Karapetyan, K., Katz, K., Liu, C., Maddipatla, Z., Malheiro, A., McDaniel, K., Ovetsky, M., Riley, G., Zhou, G., Holmes, J.B., Kattman, B.L., Maglott, D.R.: ClinVar: improving access to variant interpretations and supporting evidence. Nucleic Acids Research **46**(D1), 1062–1067 (2018) https://doi.org/10.1093/nar/gkx1153

[28] Iannella, R., Villata, S.: ODRL Information Model 2.2. W3C (2018). https://www.w3.org/TR/odrl-model/ Accessed 2025-11-27

[29] OASIS: eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard (2013). https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[30] Katayama, T., Kawashima, S., Micklem, G., Kawano, S., Kim, J.-D., Kocbek, S., Okamoto, S., Wang, Y., Wu, H., Yamaguchi, A., *et al.*: Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. F1000Research **8**, 1677 (2019) https://doi.org/10.12688/f1000research.18238.1

[31] Lindenbaum, P.: JVarkit: java-based utilities for Bioinformatics. figshare (2015). https://doi.org/10.6084/m9.figshare.1425030.v1 . https://figshare.com/articles/journal_contribution/JVarkit_java_based_utilities_for_Bioinformatics/1425030

[32] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. PLOS Computational Biology **18**(5), 1009123 (2022) https://doi.org/10.1371/journal.pcbi.1009123

[33] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation database. Human Genome Variation **9**(1), 44 (2022) https://doi.org/10.1038/s41439-022-00222-9

[34] Bodrug-Schepers, A., Chabane, H., Montoya, G., Serrano-Alvarado, P., Redon, R., Gaignard, A.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In: Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025). CEUR Workshop Proceedings, vol. 4196. CEUR Workshop Proceedings, Barcelona, Spain (2025). https://ceur-ws.org/Vol-4196/paper_16.pdf Accessed 2025-11-24

[35] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Zenodo record, version 4 (2020). https://doi.org/10.5281/zenodo.3903111

[36] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03

(2019). https://gitlab.com/roelj/sparqling-genomics

[37] Baran, J., Durgahee, B.S.B., Eilbeck, K., Antezana, E., Hoehndorf, R., Dumontier, M.: GFVO: the Genomic Feature and Variation Ontology. PeerJ **3**, 933 (2015) https://doi.org/10.7717/peerj.933

[38] Hail Team: Hail: scalable genomic data analysis. Accessed October 2026 (2026). https://github.com/hail-is/hail

[39] TileDB, Inc.: TileDB-VCF: efficient variant-call data storage and retrieval. Accessed October 2026 (2026). https://github.com/TileDB-Inc/TileDB-VCF

[40] Rambla, J., Baudis, M., Ariosa, R., *et al.*: Beacon v2 and Beacon networks: A "lingua franca" for federated data discovery in biomedical genomics, and beyond. Human Mutation **43**(6), 791–799 (2022) https://doi.org/10.1002/humu.24369

[41] Kelleher, J., Lin, M., Albach, C.H., Birney, E., *et al.*: htsget: a protocol for securely streaming genomic data. Bioinformatics **35**(1), 119–121 (2019) https://doi.org/10.1093/bioinformatics/bty492

[42] HL7 International: HL7 FHIR Genomics Reporting Implementation Guide. Accessed October 2026 (2026). https://hl7.org/fhir/uv/genomics-reporting/

[43] Touré, V., Unni, D., Krauss, P., Abdelwahed, A., Buchhorn, J., Hinderling, L., Geiger, T.R., Österle, S.: The SPHN Schema Forge – transform healthcare semantics from human-readable to machine-readable by leveraging semantic web technologies. Journal of Biomedical Semantics **16**, 9 (2025) https://doi.org/10.1186/s13326-025-00330-9

[44] Gaignard, A., Rosnet, T., Lamotte, F., Lefort, V., Devignes, M.-D.: FAIR-Checker: supporting digital resource findability and reuse with knowledge graphs and semantic web standards. Journal of Biomedical Semantics **14**, 7 (2023) https://doi.org/10.1186/s13326-023-00289-5

[45] Ansari, A., Conte, M., Flynn, A., Paturkar, A.: A prototype ETL pipeline that uses HL7 FHIR RDF resources when deploying pure functions to enrich knowledge graph patient data. Journal of Biomedical Semantics **16**, 16 (2025) https://doi.org/10.1186/s13326-025-00335-4

[46] Crum, E.: VCF Core Vocabulary. Version 2.1.3. GitHub repository. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026 (2026). https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a

[47] Crum, E., Buelens, B., Ertaylan, G., Taelman, R.: VCF Core Vocabulary: A Semantic Foundation for RDF Representations of VCF. Manuscript in preparation for SWAT4HCLS 2027; preprint at commit 9507dd9 (2026). https://github.com/ecrum19/SWAT4HCLS_2027/blob/9507dd9/long_revised/main_long.pdf

[48] VCF-RDFizer maintainers: ecrum19/vcf-rdfizer. Docker Hub image, tag 3.3.1, digest sha256:3ad71b1a54612142e3be43b2 4e7e4a38551949cb641f4421bcae9bf051102993. Published 2026-10-08 (2026). https://hub.docker.com/r/ecrum19/vcf-rdfizer

[49] VCF-RDFizer maintainers: vcf-rdfizer. Python Package Index (PyPI), version 3.3.1. Released 2026-10-08 (2026). https://pypi.org/project/vcf-rdfizer/3.3.1/

[50] VCF-RDFizer maintainers: vcf-rdfizer. conda-forge, version 3.3.1. Published 2026-10-08 (2026). https://anaconda.org/conda-forge/vcf-rdfizer

[51] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. Journal of Biomedical Semantics **7**(1), 39 (2016) https://doi.org/10.1186/s13326-016-0067-z

[52] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. Genome Biology **6**(5), 44 (2005) https://doi.org/10.1186/gb-2005-6-5-r44

[53] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In: Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025). CEUR Workshop Proceedings, Barcelona, Spain (2025). https://www.dei.unipd.it/~silvello/papers/2025_MCS_SWAT4HCLS.pdf

[54] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. Cell Genomics **1**(2), 100027 (2021) https://doi.org/10.1016/j.xgen.2021.100027

[55] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., ONeill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). https://doi.org/10.5281/zenodo.15059674 . https://doi.org/10.5281/zenodo.15059674 Accessed 2026-03-02

[56] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). https://rml.io/specs/rml/ Accessed 2025-11-24

[57] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO:

An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d'Amato, C. (eds.) The Semantic Web – ISWC 2022 vol. 13489, pp. 697–713. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40 . Series Title: Lecture Notes in Computer Science. https://link.springer.com/10.1007/978-3-031-19433-7_40 Accessed 2025-11-24

[58] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). Web Semantics: Science, Services and Agents on the World Wide Web **19**, 22–41 (2013)

[59] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: The Semantic Web – ISWC 2025. Lecture Notes in Computer Science, vol. 16141, pp. 313–331. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-09530-5_18 . https://doi.org/10.1007/978-3-032-09530-5_18

[60] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. ACM Transactions on Information Systems **37**(1), 1–30 (2019) https://doi.org/10.1145/3231935

[61] Pedersen, B.S., Quinlan, A.R.: cyvcf2: fast, flexible variant analysis with Python. Bioinformatics **33**(12), 1867–1869 (2017) https://doi.org/10.1093/bioinformatics/btx057

[62] World Wide Web Consortium: Shapes Constraint Language (SHACL). W3C Recommendation (2017). https://www.w3.org/TR/shacl/ Accessed 2026-10-02

[63] Garfinkel, S.L., Guttman, B., Near, J.P., Dajani, A.N., Singer, P.: De-identifying government datasets: Techniques and governance. NIST Special Publication 800-188, National Institute of Standards and Technology (2023). https://doi.org/10.6028/NIST.SP.800-188

[64] Global Alliance for Genomics and Health: Data Security Infrastructure Policy (DSIP), Version 4.0. 25 September 2019 (2019). https://github.com/ga4gh/data-security/blob/master/DSIP/DSIP_v4.0.md

[65] Personal Genome Project (2005). https://my.pgp-hms.org/public_genetic_data Accessed 2025-11-24

[66] NIST Genome in a Bottle (2025). https://www.nist.gov/programs-projects/genome-bottle Accessed 2025-11-24

[67] The 1000 Genomes Project Consortium: A global reference for human genetic variation. Nature **526**(7571), 68–74 (2015) https://doi.org/10.1038/nature15393

[68] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. Science **372**(6537), 7117 (2021) https://doi.org/10.1126/science.abf7117

[69] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small variants with linked and long reads. Cell Genomics **2**(5), 100128 (2022) https://doi.org/10.1016/j.xgen.2022.100128

[70] Bast, H., Buchhold, B.: QLever: A query engine for efficient SPARQL+Text search. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM), pp. 647–656 (2017). https://doi.org/10.1145/3132847.3132921 . https://doi.org/10.1145/3132847.3132921

[71] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular SPARQL query engine for the web. In: Proceedings of the 17th International Semantic Web Conference, pp. 239–255. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-00668-6_15 . https://comunica.dev/

[72] Comunica Association: Querying local files from the command line (2026). https://comunica.dev/docs/query/getting_started/query_cli_file/ Accessed 2026-08-31

[73] Comunica Association: HDT: Querying HDT files with Comunica (2026). https://comunica.dev/docs/query/advanced/hdt/ Accessed 2026-08-31

[74] Crum, E.: Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas` (2026). https://github.com/ecrum19/comunica-feature-cottas

[75] Miller, D.T., Lee, K., Abul-Husn, N.S., Amendola, L.M., *et al.*: ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). Genetics in Medicine **25**(8), 100866 (2023) https://doi.org/10.1016/j.gim.2023.100866

[76] Byrska-Bishop, M., Evani, U.S., Zhao, X., *et al.*: High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. Cell **185**(18), 3426–344019 (2022) https://doi.org/10.1016/j.cell.2022.08.004

[77] National Center for Biotechnology Information: ClinVar VCF Release (GRCh38). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/

[78] Crum, E., Buelens, B., Taelman, R., Ertaylan, G.: VCF-RDFizer evaluation:

benchmark harness and run records. Zenodo. Dataset, published 2026-10-11. All versions: https://doi.org/10.5281/zenodo.23301185 (2026). https://doi.org/10.5281/zenodo.23301186 . https://doi.org/10.5281/zenodo.23301186

[79] Anthropic: Claude Opus 5.5 (Extra) [Large language model]. Accessed through the Claude Code desktop application. Used September 2026 (2026). https://www.anthropic.com/claude

[80] OpenAI: GPT-6 Astra (X-High) [Large language model]. Accessed through the Codex desktop application. Used September 2026 (2026). https://openai.com/